Fine-Tuning Bengali Tesseract Model for New Conjunct Characters

---

Fine-Tuning Bengali Tesseract OCR for Ligatures (যুক্তাক্ষর)

In this guide, I'll walk you through the process of fine-tuning the Bengali Tesseract OCR model to better recognize conjunct characters (ligatures), which are often a challenge for OCR systems. This process focuses on adding complex conjuncts, improving the training dataset, and optimizing the model for better recognition of these ligatures.

Table of Content:

1. Prerequisites
2. Working of Tesseract OCR
3. Data Preparation
    - Explicitly Include Conjuncts in Ground Truth
    - Generate Synthetic Images for Ligatures
    - Create .box and .lstmf
4. LSTM Architecture and Training
5. Fine-Tuning with LSTM
    - Training the Model
6. Advanced Techniques for Ligature Handling
    - Modify unicharset
    - Custom danambigs and ambigs
    - Augment Wordlist
7. Evaluation
8. Comparing Results
9. Conclusion

1. Prerequisites

Before I dive into the actual process, there are a few things I need to have in place:

- **Tesseract OCR with LSTM support**: Make sure I have the latest Tesseract version installed with LSTM (Long Short-Term Memory) support enabled.
- **Synthetic images of Bengali text**: These images should specifically feature common and complex conjunct characters.
- **Ground truth files**: These contain the exact text for each of my synthetic images.
- **Tesseract training tools**: I'll need tools like lstmtraining, unicharset_extractor, combine_tessdata, and wordlist2dawg.

---

2. Working of Tesseract OCR

Tesseract is an open-source Optical Character Recognition (OCR) engine.

Originally developed by Hewlett-Packard (1984–1994), it was open-sourced by HP and Google in 2005. Since then, it has grown into one of the most accurate free OCR systems, supporting over 100 languages and a range of scripts.

Over its lifetime, Tesseract evolved through two main architectural eras:

- Tesseract 3.x and earlier (classic heuristic-based engine)
- Tesseract 4.x and 5.x (deep learning-based LSTM engine)

Here we describe both architectures and explains in detail how to train modern models.

Classic Tesseract (3.x) Architecture

Input Requirements

- Binary (black/white) images.

- Optional polygonal text region definitions.

Unlike later OCR systems, Tesseract did not have its own sophisticated page layout analysis. It relied on external tools or user-provided regions.

Pipeline Overview

Step 1: Connected Component Analysis

- The image is analyzed to find blobs (connected pixel clusters).
- For each blob:
    - Outer and nested outlines are stored.
    - The nesting allows identification of holes (e.g., the center of “o”) and even white-on-black text.

Step 2: Grouping Blobs into Lines

- Blobs are sorted horizontally.
- Assigned to lines by comparing vertical overlap and slope.
- Tesseract used a least median of squares fit to model the baseline of each text line.

Step 3: Baseline Fitting

- Curved baselines (common in scanned book pages) are fitted using quadratic splines.

Step 4: Fixed vs. Proportional Text Detection

- Lines are classified as:
    - Fixed pitch (monospaced): e.g., typewriter text.
    - Proportional (variable spacing): most printed text.
- In fixed-pitch lines, characters are segmented by pitch measurement.
- In proportional text, spaces are identified by measuring gaps between blobs.

Word Recognition Process

Recognition in Tesseract 3.x followed a two-pass process:

Pass 1

- For each word:
    - Attempt segmentation into characters.
    - Use a static classifier to recognize characters.
- If recognition is confident, the result is fed into the adaptive classifier, which learns document-specific prototypes.

Pass 2

- Words with poor recognition are reprocessed, using the adaptive classifier to improve accuracy.

Handling Difficult Cases

Chopping joined characters

- When segmentation is poor, Tesseract identifies concave points on the blob outline.
- Candidate splits (chops) are generated and tested.
- Chops are prioritized based on estimated improvement to recognition confidence.

Associating broken characters

- If characters were over-segmented, Tesseract performed an A* search over possible segment combinations.
- Each combination was evaluated by re-classifying merged blobs.

Classifier Details

Static Classifier

- Feature extraction: Short line segments along the outline.
- Each feature: (x, y, angle)
- Prototypes: Clustered templates representing each character class.

Classification Steps

1. Class Pruning
    - A coarse lookup table yields candidate character classes.

2. Detailed Similarity
    - Each unknown feature is matched to prototype features.
    - A similarity score (distance) is computed.
3. Confidence Scoring
    - Ratings per class, combined across configurations.

Adaptive Classifier

- Uses same classifier code but trained dynamically on the current document.
- Improves recognition of consistent fonts and styles.

Linguistic Analysis

- Minimal dictionary or language modeling.
- When a segmentation is considered, several hypotheses are evaluated:
    - Frequent word list
    - Dictionary
    - Upper/lowercase variants
    - Numeric words
- Final choice is based on combined distance ratings.

Limitations of Classic Tesseract

- No robust handling of curved layouts or complex scripts.
- Heavy reliance on heuristics.
- Lacked modern language modeling or probabilistic decoders.
- Difficult to adapt to handwriting or noisy scans.

Modern Tesseract Architecture (4.x / 5.x)

Tesseract 4.x (2017) and 5.x (2021+) introduced a major leap:

Replaced the classic classifier with a deep learning-based LSTM recognizer.

Architecture is now hybrid:

- Layout Analysis: Still uses traditional methods (blobs, lines).
- Text Recognition: Neural network with CNN + Bidirectional LSTM + CTC.

Pipeline Overview

Image Preprocessing

- Grayscale conversion.
- Thresholding (Otsu).
- Deskewing.
- Noise removal.

Layout Analysis

- Connected component analysis.
- Baseline detection.
- Line segmentation.

Recognition Engine

- Each line is processed by a deep neural network:
    - Shallow CNN: Extracts visual features.
    - Bidirectional LSTM stack: Learns character sequences.
    - CTC decoder: Converts predictions into text.

Language Model

- Dictionary or frequency-based correction.

Neural Network Details

Components:

- Input: Line images.
- CNN Layers: Small convolutional extractor.
- LSTM Layers: Multiple bidirectional LSTM layers (usually 2–3).

- Softmax: Outputs per time step (one per character class).
- CTC: (Connectionist Temporal Classification)
    - Aligns sequences without explicit segmentation.

Model File (.traineddata)

Tesseract's model file includes:

- Network weights.
- Character set (unicharset).
- Dictionaries and frequency lists.
- Parameters and metadata.

Benefits of the LSTM Pipeline

- High accuracy even on degraded images.
- Support for diverse scripts.
- No need for per-character segmentation heuristics.
- Easier training on new languages.

3. Data Preparation

Explicitly Include Conjuncts in Ground Truth

One of the biggest challenges with Bengali OCR is handling ligatures (connected characters). These characters are often rare in the original training dataset, and Tesseract might struggle to recognize them.

To help solve this, I'll curate a dataset where these complex conjunct characters appear prominently. I'll also make sure the Unicode sequences are accurately represented in my .gt.txt files.

For example, some words with common conjuncts are:

শিক্ষা, স্থগিত, বৃষ্টিপাত, গ্রীষ্মকাল, রক্তচাপ

These contain conjuncts like **ক্ষ**, **ষ্ঠ**, **ক্ক**, **ত্ব**, and **গ্র**.

What I do:

- I ensure these conjuncts are included throughout my training set.
- I write the exact Unicode sequences in the .gt.txt files—no errors, no substitutions.

Generate Synthetic Images for Ligatures

Now, I'll generate synthetic images that specifically focus on these conjuncts. For this, I can use various tools:

- **TextRecognitionDataGenerator**: A great tool for generating synthetic datasets.
- **Custom Scripts**: I can use libraries like PIL or OpenCV to render Bengali text with custom fonts.
- **Font Variety**: I'll make sure to use Bengali-rich fonts like Siyam Rupali, SolaimanLipi, Nikosh, and Likhan. I'll also vary the font style and size.

To ensure quality, I'll render images at high resolutions (300 DPI or higher). I'll also add noise or artifacts to simulate real-world conditions like scanned documents or low-quality printing.

What I do:

- Render the images in various fonts, focusing on ligatures.
- Include noise or scanning artifacts if needed.

Create .box and .lstmf Files

Once I have the synthetic images, I need to create .box and .lstmf files. The .box file contains character bounding boxes, and the .lstmf file is required for LSTM-based training.

1. **Generate the .box File**:

I use the following command to generate .box files from my images:

```
cd /mnt/c/Users/sr422/Downloads/cleaned_files || { echo "Directory not found!"; exit 1; }

lang=ben  # change to your traineddata language code if needed
psm=6     # adjust Page Segmentation Mode if needed

for tif in *.tif; do
  base="${tif%.tif}"
  gt="${base}.gt.txt"

  if [[ -f "$gt" ]]; then
    echo "Generating $base.box ..."
    tesseract "$tif" "$base" -l $lang --psm $psm makebox
  else
    echo "Skipping $tif — $gt not found."
  fi
done
```

2. **Generate the .lstmf File**:

After creating the .box file, I generate the .lstmf file using this command:

```
cd /mnt/c/Users/sr422/Downloads/cleaned_files || { echo "Directory not found!"; exit 1; }

lang=ben   # Replace with your language code if needed
psm=6      # Page segmentation mode

for tif in *.tif; do
  base="${tif%.tif}"
  gt="${base}.gt.txt"

  if [[ -f "$gt" ]]; then
    echo "Generating $base.lstmf ..."
    tesseract "$tif" "$base" --psm $psm -l $lang lstm.train
  else
    echo "Skipping $tif — $gt not found."
  fi
done
```

This will produce output_base.lstmf that I'll use for LSTM training.

4. LSTM Architecture and Training

High-Level Architecture Overview

An LSTM cell replaces the standard RNN cell.

Core components:

- Cell state (c_t): This is the "memory" of the network. It flows mostly unchanged through time.
- Hidden state (h_t): This is the output at each time step.
- Gates: Structures that control the flow of information.

At each time step t, the LSTM processes:

- Current input vector x_t
- Previous hidden state h_{t-1}
- Previous cell state c_{t-1}

LSTM Cell Internals

The **LSTM cell** has **three main gates** and **one candidate memory update**:

Forget Gate (f_t)

- Decides **what information to forget** from the cell state.
- Equation:

  $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$

- Values in f_t are between 0 and 1 (0 = forget, 1 = keep).

Input Gate (i_t)

- Decides **which new information to store**.
- Equation:

  $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$

Candidate Memory (ĉ_t)

- Proposes **new candidate values** to be added to the cell state.
- Equation:

  c^t=tanh(Wc·[ht−1,xt]+bc)

Update Cell State

- Combine previous memory (scaled by forget gate) and new memory (scaled by input gate).
- Equation:

  ct=ft⊙ct−1+it⊙c^t

Output Gate (o_t)

- Decides **what to output** from the cell.
- Equation:

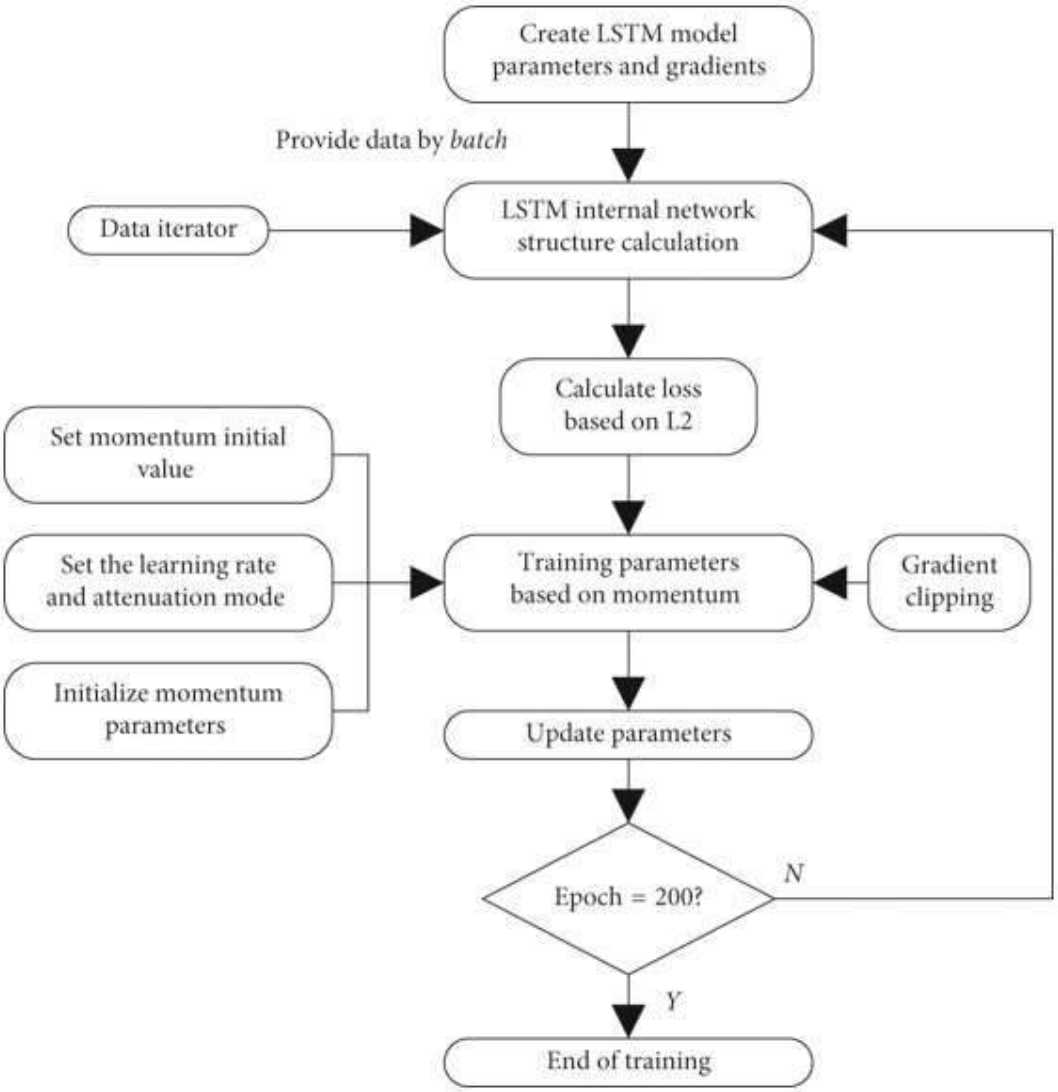
  ot=σ(Wo·[ht−1,xt]+bo)

Hidden State Update

- Output combines output gate and transformed cell state.
- Equation:

  ht=ot⊙tanh(ct)

Diagrammatic Representation of LSTM-based training



Training Process (LSTM Training Pipeline)

The training follows the general supervised learning pipeline, but applied to **sequences**.

- **Step 1: Forward Pass**

For each input sequence:

- Initialize c_0 and h_0 (often zeros).
- For each timestep t in the sequence:
    - Compute the gates and updated states as above.
    - Produce output h_t (either directly or via an output layer).

🔹 **Step 2: Loss Computation**

- Depending on the task:
    - Sequence labeling: compute loss at every timestep.
    - Sequence classification: compute loss at the end.
    - Sequence-to-sequence: compute per-step losses over target sequence.

🔹 **Step 3: Backpropagation Through Time (BPTT)**

- Compute gradients by **unrolling the LSTM over time** and backpropagating errors.
- Gradients flow through:
    - Cell states (c_t)
    - Hidden states (h_t)
    - Gate activations
- This is where LSTMs shine: the cell state c_t allows **better gradient flow**, mitigating vanishing gradients.

🔹 **Step 4: Parameter Update**

- Using optimizers (SGD, Adam, RMSprop), update all weight matrices:
    - Wf, Wi, Wc, Wo
    - And corresponding biases.

5. Fine-Tuning with LSTM

Now, I'm ready to fine-tune the Tesseract model. I'll use the LSTM-based training mode to refine the model.

Training the Model

Start LSTM Training:

I use the following command to fine-tune the model:

```
mkdir -p test_lstmf_logs

for f in *.lstmf; do
  echo "$PWD/$f" > single_file.txt

  echo "Testing $f ..."
  lstmtraining \
    --continue_from /mnt/c/tesseract/tessdata/ben_traindata/ben.lstm \
    --model_output test_lstmf_logs/temp_model \
    --train_listfile single_file.txt \
    --traineddata /mnt/c/tesseract/tessdata/ben_traindata/ben.traineddata \
    --max_iterations 1 \
    --debug_interval 0

  status=$?
  if [[ $status -ne 0 ]]; then
    echo "❌ $f caused a crash or failed."
  else
    echo "✅ $f looks good."
  fi
done
```

6. Advanced Techniques for Ligature Handling

Modify unicharset

The **unicharset** is a key component of the model—it defines the character set Tesseract can recognize. If my fine-tuning dataset includes new conjuncts that aren't in the original ben.unicharset, the model won't learn them.

1. **Extract unicharset** from my training data:
2. unicharset_extractor your-truth.txt
3. **Merge with the existing unicharset** (if needed):
4. merge_unicharsets original_ben.unicharset your_new.unicharset

What I do:

- I check that my conjuncts are being treated as separate grapheme clusters in the unicharset.

Custom dangambigs and ambigs Files

If Tesseract frequently misrecognizes ligatures (e.g., splitting **ক্ত** into **ক্** and **ত**), I can provide custom ambiguity rules through the **dangambigs** and **ambigs** files.

For example, I might add a rule like this:

ক্ ত যুক্ত 1

This tells Tesseract to interpret **ক্ + ত** as **ক্ত**.

What I do:

- I add specific rules to handle common misrecognitions, especially those involving conjuncts.

Augment the Wordlist

Tesseract's wordlist (ben.wordlist) can be augmented with a large collection of Bengali words. A well-rounded wordlist helps Tesseract better recognize words, especially those containing rare or complex ligatures.

1. **Rebuild the wordlist's .dawg file**:

   wordlist2dawg ben.wordlist ben.lstm-word-dawg ben.unicharset

2. **Rebuild other .dawg files** (if needed):

   wordlist2dawg ben.punc ben.lstm-punc-dawg ben.unicharset

   wordlist2dawg ben.numbers ben.lstm-number-dawg ben.unicharset

3. **Repackage trained data**:

   combine_tessdata -o ben.traineddata \

4. ben_.lstm \
5. ben_.lstm-word-dawg \
6. ben_.lstm-punc-dawg \
7. ben_.lstm-number-dawg \
8. ben_.lstm-unicharset \
9. ben_.lstm-recoder \
10. ben_.config \
11. ben_.version

What I do:

- I make sure to augment the wordlist with common Bengali words, including compound words, and rebuild the .dawg files.

---

7. Evaluation

Once I've fine-tuned my model, I'll need to evaluate its performance. I'll run Tesseract on a test set of images with ligatures:

tesseract test_image.png output_text -l new_bengali_model

Then, I'll compare the OCR output to the ground truth and calculate the **Word Error Rate (WER)**.

---

8. Comparing Results

To compare the old and new models:

1. **Old Model**:

   tesseract test_image.png old_model_output -l bengali

2. **New Model**:

   tesseract test_image.png new_model_output -l new_bengali_model

3. **Comparison:**

   Image 1 (Ananda 1)

Old Model Result:

Total words in ground truth: 2986

Correctly detected words: 2302

Simplified Word Error Rate: 0.2291

New Model Result:

Total words in ground truth: 2986

Correctly detected words: 2631

Simplified Word Error Rate: 0.1189

Image 2 (Ananda 2)

Old Model Result:

Total words in ground truth: 3397

Correctly detected words: 3056

Simplified Word Error Rate: 0.1004

New Model Result:

Total words in ground truth: 3397

Correctly detected words: 3218

Simplified Word Error Rate: 0.0527

Image 3 (Ananda 3)

Old Model Result:

Total words in ground truth: 3142

Correctly detected words: 2510

Simplified Word Error Rate: 0.2011

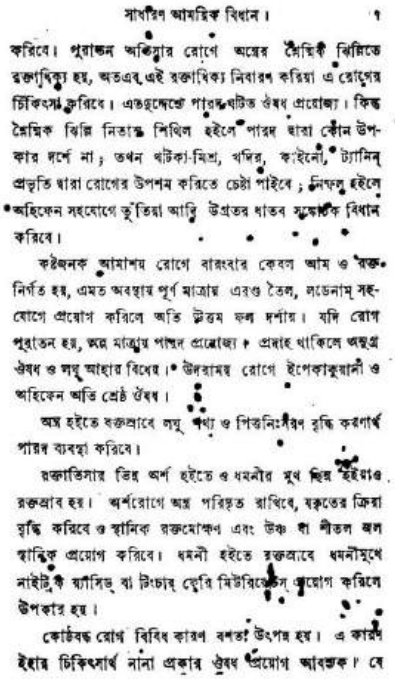
New Model Result:

Total words in ground truth: 3142

Correctly detected words: 2919

Simplified Word Error Rate: 0.071

Image 4 (Storybook 1)



সাধারণ আময়িক বিধান। ৭

করিবে। পুরাতন অজীর্ণ রোগে অন্নের মৈহিক ক্লিন্নিতে রক্তাধিক্য হয়, অতএব এই রক্তাধিক্য নিবারণ করিয়া এ রোগের চিকিৎসা করিবে। এতদুদ্দেশ্যে পারদঘটিত ঔষধ প্রয়োজ্য। কিন্তু মৈহিক ক্লিন্নি নিতান্ত শিথিল হইলে পারদ দ্বারা কোন উপকার দর্শে না; তখন খটিকা-মিশ্র, খদির, কাইনো, ট্যানিন্ প্রভৃতি দ্বারা রোগের উপশম করিতে চেষ্টা পাইবে; নিষ্ফল হইলে অহিফেন সহযোগে তুঁতিয়া আদি উগ্রতর ধাতব সুদাহক বিধান করিবে।

কষ্টজনক আমাশয় রোগে বারংবার কেবল আম ও রক্ত নির্গত হয়, এমত অবস্থায় পূর্ণ মাত্রায় এরণ্ড তৈল, লডেনাম্ সহযোগে প্রয়োগ করিলে অতি উত্তম ফল দর্শায়। যদি রোগ পুরাতন হয়, অল্প মাত্রায় পাশুন প্রয়োজ্য। প্রদাহ থাকিলে অনুগ্র ঔষধ ও লঘু আহার বিধেয়। উদরাময় রোগে ইপেকাকুয়ানী ও অহিফেন অতি শ্রেষ্ঠ ঔষধ।

অম্ল হইতে রক্তস্রাবে লঘু পথ্য ও পিত্তনিঃসারণ বৃদ্ধি করণার্থ পারদ ব্যবস্থা করিবে।

রক্তাতিসার ভিন্ন অর্শ হইতে ও ধমনীর মুখ ছিন্ন হইয়াও রক্তস্রাব হয়। অর্শরোগে অন্ন পরিষ্কৃত রাখিবে, যকৃতের ক্রিয়া বৃদ্ধি করিবে ও স্থানিক রক্তমোক্ষণ এবং উষ্ণ বা শীতল জল স্থানিক প্রয়োগ করিবে। ধমনী হইতে রক্তস্রাবে ধমনীমুখে নাইট্রিক ষ্যাসিড্ বা টিংচার্ ফেরি মিউরিয়েটিস প্রয়োগ করিলে উপকার হয়।

কোষ্ঠবদ্ধ রোগ বিবিধ কারণ বশতঃ উৎপন্ন হয়। এ কারণ ইহার চিকিৎসার্থ নানা প্রকার ঔষধ প্রয়োগ আবশ্যক। যে

Old Model Result:

Total words in ground truth: 193

Correctly detected words: 128

Simplified Word Error Rate: 0.3368

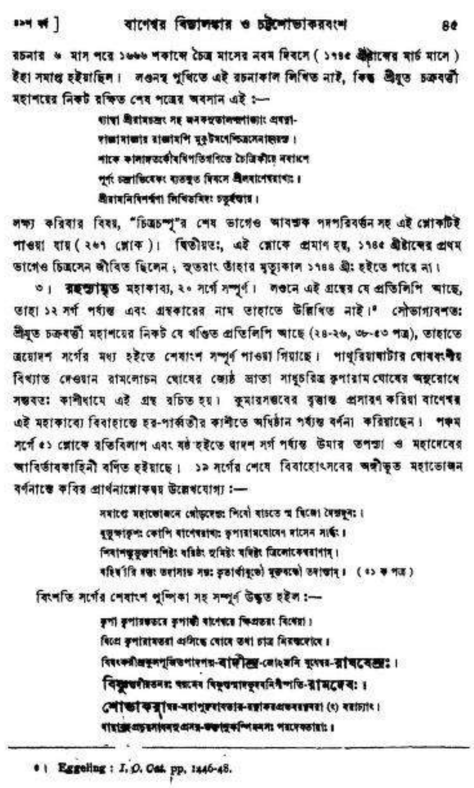
New Model Result:

Total words in ground truth: 193

Correctly detected words: 139

Simplified Word Error Rate: 0.2798

Image 5 (Storybook 2)



বাণেশ্বর বিদ্যালঙ্কার ও চক্রশোভাকরবংশ ৪৫

রচনার ৬ মাস পরে ১৬৬৬ শকাব্দে চৈত্র মাসের নবম দিবসে (১৭৪৫ খ্রীষ্টাব্দের মার্চ মাসে) ইহা সমাপ্ত হইয়াছিল। লণ্ডনস্থ পুথিতে এই রচনাকাল লিখিত নাই, কিন্তু শ্রীযুত চক্রবর্ত্তী মহাশয়ের নিকট রক্ষিত শেষ পত্রের অবসান এই :—

লক্ষ্য করিবার বিষয়, "চিত্রচম্পূ"র শেষ ভাগেও আবশ্যক পদপরিবর্ত্তন সহ এই শ্লোকটিই পাওয়া যায় (২৬৭ শ্লোক)। দ্বিতীয়তঃ, এই শ্লোকে প্রমাণ হয়, ১৭৪৫ খ্রীষ্টাব্দের প্রথম ভাগেও চিত্রসেন জীবিত ছিলেন ; সুতরাং তাঁহার মৃত্যুকাল ১৭৪৪ খ্রীঃ হইতে পারে না।

৩। **রহস্যামৃত** মহাকাব্য, ২০ সর্গে সম্পূর্ণ। লণ্ডনে এই গ্রন্থের যে প্রতিলিপি আছে, তাহা ১২ সর্গ পর্য্যন্ত এবং গ্রন্থকারের নাম তাহাতে উল্লিখিত নাই।* সৌভাগ্যবশতঃ শ্রীযুত চক্রবর্ত্তী মহাশয়ের নিকট যে খণ্ডিত প্রতিলিপি আছে (২৪-২৬, ৩৮-৫৩ পত্র), তাহাতে ত্রয়োদশ সর্গের মধ্য হইতে শেষাংশ সম্পূর্ণ পাওয়া গিয়াছে। পাথুরিয়াঘাটার ঘোষবংশীয় বিখ্যাত দেওয়ান রামলোচন ঘোষের জ্যেষ্ঠ ভ্রাতা সাধুচরিত্র কৃপারাম ঘোষের অনুরোধে সম্ভবতঃ কাশীধামে এই গ্রন্থ রচিত হয়। কুমারসম্ভবের বৃত্তান্ত প্রসারণ করিয়া বাণেশ্বর এই মহাকাব্যে বিবাহান্তে হর-পার্ব্বতীর কাশীতে অধিষ্ঠান পর্য্যন্ত বর্ণনা করিয়াছেন। পঞ্চম সর্গে ৫১ শ্লোকে রতিবিলাপ এবং ষষ্ঠ হইতে দ্বাদশ সর্গ পর্য্যন্ত উমার তপস্যা ও মহাদেবের আবির্ভাবকাহিনী বর্ণিত হইয়াছে। ১৯ সর্গের শেষে বিবাহোৎসবের অঙ্গীভূত মহাভোজন বর্ণনান্তে কবির প্রার্থনাশ্লোকদ্বয় উল্লেখযোগ্য :—

বিংশতি সর্গের শেষাংশ পুষ্পিকা সহ সম্পূর্ণ উদ্ধৃত হইল :—

* । Eggeling : *I. O. Cat.* pp. 1446-48.

Old Model Result:

Total words in ground truth: 259

Correctly detected words: 53

Simplified Word Error Rate: 0.7954

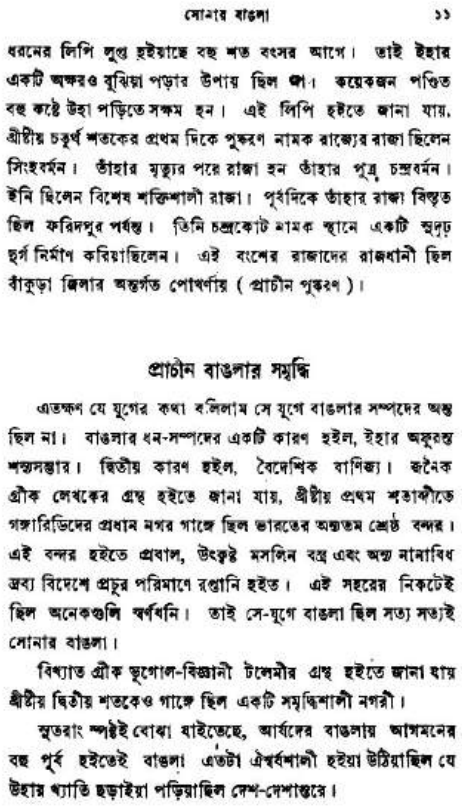
New Model Result:

Total words in ground truth: 259

Correctly detected words: 107

Simplified Word Error Rate: 0.5869

Image 6 (Storybook 3)



সোনার বাঙলা ১১

ধরনের লিপি লুপ্ত হইয়াছে বহু শত বৎসর আগে। তাই ইহার একটি অক্ষরও বুঝিয়া পড়ার উপায় ছিল না। কয়েকজন পণ্ডিত বহু কষ্টে উহা পড়িতে সক্ষম হন। এই লিপি হইতে জানা যায়, খ্রীষ্টীয় চতুর্থ শতকের প্রথম দিকে পুষ্করণ নামক রাজ্যের রাজা ছিলেন সিংহবর্মন। তাঁহার মৃত্যুর পরে রাজা হন তাঁহার পুত্র চন্দ্রবর্মন। ইনি ছিলেন বিশেষ শক্তিশালী রাজা। পূর্বদিকে তাঁহার রাজ্য বিস্তৃত ছিল ফরিদপুর পর্যন্ত। তিনি চন্দ্রকোট নামক স্থানে একটি সুদৃঢ় দুর্গ নির্মাণ করিয়াছিলেন। এই বংশের রাজাদের রাজধানী ছিল বাঁকুড়া জিলার অন্তর্গত পোখর্ণায় (প্রাচীন পুষ্করণ)।

প্রাচীন বাঙলার সমৃদ্ধি

এতক্ষণ যে যুগের কথা বলিলাম সে যুগে বাঙলার সম্পদের অন্ত ছিল না। বাঙলার ধন-সম্পদের একটি কারণ হইল, ইহার অফুরন্ত শস্যসম্ভার। দ্বিতীয় কারণ হইল, বৈদেশিক বাণিজ্য। জনৈক গ্রীক লেখকের গ্রন্থ হইতে জানা যায়, খ্রীষ্টীয় প্রথম শতাব্দীতে গঙ্গারিডিদের প্রধান নগর গাঙ্গে ছিল ভারতের অন্যতম শ্রেষ্ঠ বন্দর। এই বন্দর হইতে প্রবাল, উৎকৃষ্ট মসলিন বস্ত্র এবং অন্য নানাবিধ দ্রব্য বিদেশে প্রচুর পরিমাণে রপ্তানি হইত। এই সহরের নিকটেই ছিল অনেকগুলি স্বর্ণখনি। তাই সে-যুগে বাঙলা ছিল সত্য সত্যই সোনার বাঙলা।

বিখ্যাত গ্রীক ভূগোল-বিজ্ঞানী টলেমীর গ্রন্থ হইতে জানা যায় খ্রীষ্টীয় দ্বিতীয় শতকেও গাঙ্গে ছিল একটি সমৃদ্ধিশালী নগরী।

সুতরাং স্পষ্টই বোঝা যাইতেছে, আর্যদের বাঙলায় আগমনের বহু পূর্ব হইতেই বাঙলা এতটা ঐশ্বর্যশালী হইয়া উঠিয়াছিল যে উহার খ্যাতি ছড়াইয়া পড়িয়াছিল দেশ-দেশান্তরে।

Old Model Result:

Total words in ground truth: 201

Correctly detected words: 167

Simplified Word Error Rate: 0.1692

New Model Result:

Total words in ground truth: 201

Correctly detected words: 181

Simplified Word Error Rate: 0.0995

Comparison Table:

Document	No. of Words in ground-truth	No. of words in old ocr output	No. of words in new ocr output	Old WER	New WER
Ananda 1	2986	2302	2631	0.2291	0.1189
Ananda 2	3397	3056	3218	0.1004	0.0527
Ananda 3	3142	2510	2919	0.2011	0.071
Storybook 1	193	128	139	0.3368	0.2798
Storybook 2	259	53	107	0.7954	0.5869
Storybook 3	201	167	181	0.1692	0.0995

Refer for WER code:
https://colab.research.google.com/drive/1X45sbOoHR099ylBvK4eLJ6RrlIICDhcc?usp=sharing

9. Conclusion

By following these steps, I’ve significantly improved Tesseract's recognition of Bengali conjunct characters. Fine-tuning the model with carefully curated datasets, expanding the wordlist, and applying advanced techniques like modifying the unicharset and adding custom ambiguity rules has helped create a much more robust OCR system.